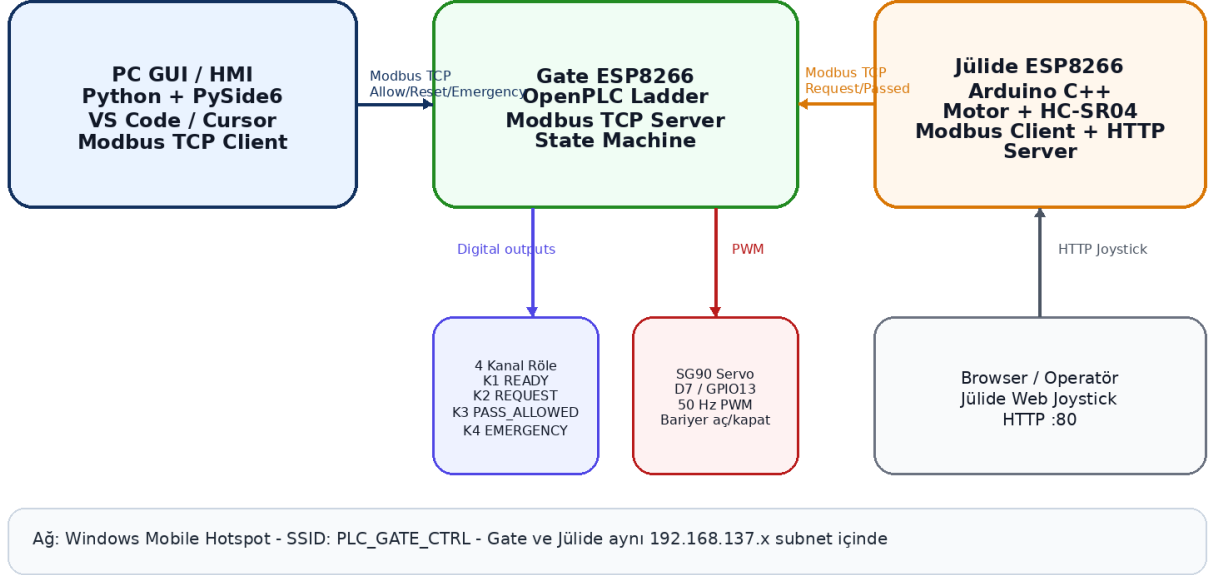


Jülide Robot + OpenPLC Gate Automation System

ESP8266 tabanlı mobil robot, OpenPLC Ladder gate kontrolcüsü ve PySide6 HMI entegrasyonu

Jülide Robot + OpenPLC Gate Automation System

Üç ana bileşen: PC HMI, Gate ESP/OpenPLC, Jülide ESP/Robot



Şekil 1 - Projenin genel mimarisi

Bu rapor, çalışan final prototipin donanım bağlantılarını, yazılım mimarisini, Modbus adres haritasını, OpenPLC Ladder mantığını, Jülide robot akışını, servo PWM çözümünü ve geliştirme sırasında karşılaşılan hataların çözüm kayıtlarını içerir.

Tarih: 30 Mayıs 2026

İçindekiler

- Proje özeti ve hedef
- Sistem mimarisi ve ağ topolojisi
- Malzeme listesi ve minimum gereksinimler
- Gate ESP8266 / OpenPLC donanım bağlantıları
- Jülide robot elektronik bağlantıları
- Yazılım bileşenleri ve kullanılan IDE'ler
- OpenPLC Ladder state machine
- Modbus TCP coil haritası
- Servo PWM entegrasyonu ve final çözüm
- Jülide robot çalışma mantığı
- GUI/HMI çalışma mantığı
- Karşılaşılan hatalar ve çözümler
- Kurulum, çalıştırma ve final test prosedürü
- Troubleshooting ve ders çıkarımları

1. Proje Özeti

Bu proje, küçük ölçekli bir endüstriyel otomasyon senaryosunu robotik sistemle birleştirir. Jülide adlı ESP8266 tabanlı mobil robot, bir kapı/gate noktasına yaklaşır, OpenPLC ile çalışan Gate ESP8266 kontrolcüsünden geçiş izni ister, operatör PC üzerindeki HMI/GUI üzerinden onay verir, servo bariyer açılır, robot geçer ve geçiş tamamlandığında gate tekrar READY durumuna dönerek servo bariyeri kapatır.

Proje özellikle üç konuyu aynı anda göstermektedir: PLC mantığının Ladder Diagram ile kurulması, mobil robotun sensör ve motor kontrolü, iki gömülü cihazın Modbus TCP üzerinden haberleşmesi. Böylece ders kapsamında istenen mikrodenetleyici tabanlı PLC sistemi, gerçek bir robotik senaryo ile entegre edilmiştir.

1.1 Temel hedef

- Gate tarafında OpenPLC Editor ile gerçek Ladder Diagram kullanmak.
- Jülide robotu Arduino IDE ile bağımsız mobil robot olarak çalıştırmak.
- PC üzerinde PySide6 tabanlı HMI/GUI ile operatör kontrolü sağlamak.
- HTTP tabanlı eski yaklaşımı PLC mantığına uygun Modbus TCP coil yapısına dönüştürmek.
- Röle, servo ve robot hareketlerini aynı state machine üzerinde tutarlı şekilde yönetmek.

1.2 Final çalışan akış

```
READY
  K1 READY yanar
  Servo kapalıdır

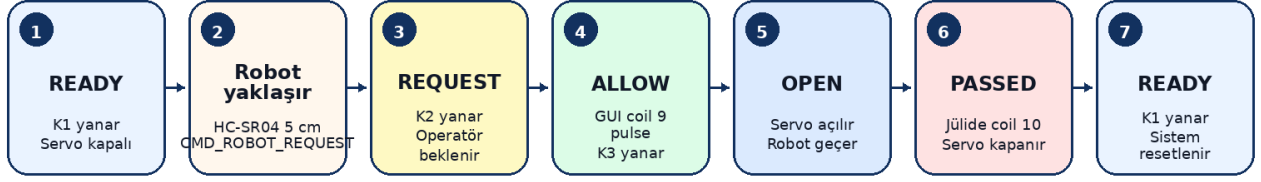
Jülide kapıya yaklaşır
  HC-SR04 mesafeyi ölçer
  5 cm altında durur
  CMD_ROBOT_REQUEST gönderir

REQUEST
  K2 REQUEST yanar
  Operatör GUI üzerinden Allow verir

PASS_ALLOWED
  K3 PASS_ALLOWED yanar
  Servo açılır
  Jülide geçer

ROBOT_PASSED
  Jülide CMD_ROBOT_PASSED gönderir
  K1 READY yanar
  Servo kapanır
```

Final Çalışma Sekansı



Önemli sonuç: Geçiş sonunda yalnızca state READY'ye dönmek yetmedi; CMD_ROBOT_PASSED sinyali ile servo close PWM komutu ayrıca gönderildi.

Bu yüzden final servo ladder mantığı: M_READY -> close, M_PASS_ALLOWED -> open, CMD_ROBOT_PASSED -> close.

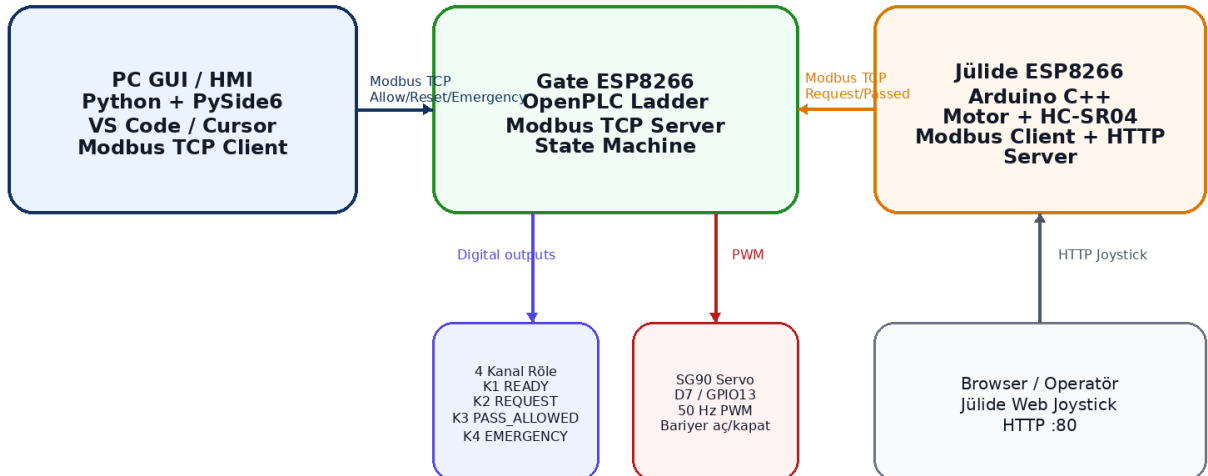
Şekil 2 - Final çalışma sekansı: READY -> REQUEST -> PASS_ALLOWED -> READY

2. Sistem Mimarisi

Sistem üç ana yazılım/donanım bloğundan oluşur: Gate ESP8266, Jülide ESP8266 ve PC GUI/HMI. Gate ESP8266, OpenPLC Ladder programını çalıştıran Modbus TCP server'dır. Jülide, Arduino C++ firmware ile çalışan mobil robot ve Modbus TCP client'tır. PC GUI ise operatör arayüzüdür ve gate ile Modbus TCP üzerinden haberleşir.

Jülide Robot + OpenPLC Gate Automation System

Üç ana bileşen: PC HMI, Gate ESP/OpenPLC, Jülide ESP/Robot



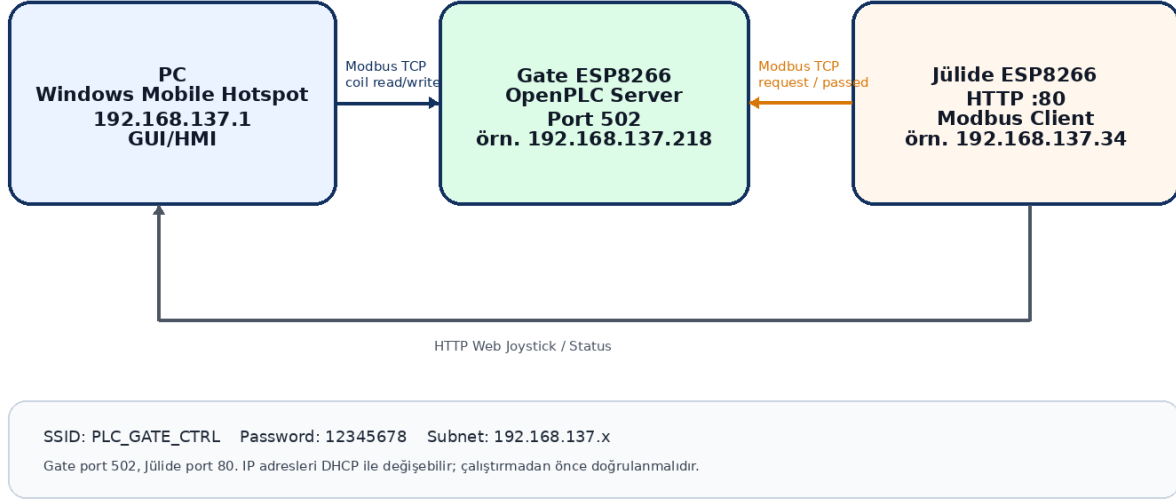
Ağ: Windows Mobile Hotspot - SSID: PLC_GATE_CTRL - Gate ve Jülide aynı 192.168.137.x subnet içinde

Şekil 3 - Üç bileşenli sistem mimarisi

2.1 Ağ topolojisi

PC, Windows Mobile Hotspot açarak yerel ağı oluşturur. Gate ESP ve Jülide aynı hotspot'a bağlanır. Bu yapı, ek router kullanmadan tamamen taşınabilir bir demo ortamı sağlar.

Ağ ve Haberleşme Topolojisi



Şekil 4 - Windows Mobile Hotspot, Modbus TCP ve HTTP haberleşme topolojisi

Bileşen	Rol	Protokol	Örnek IP / Port
PC	Hotspot + GUI/HMI	Modbus TCP client, HTTP client	192.168.137.1
Gate ESP8266	OpenPLC kontrolcü	Modbus TCP server	192.168.137.218 : 502
Jülide ESP8266	Robot kontrolcü	Modbus TCP client + HTTP server	192.168.137.34 : 80

Not: IP adresleri DHCP ile değişebilir. Çalıştırmadan önce Windows Mobile Hotspot bağlı cihazlar listesinden veya ping/Test-NetConnection ile doğrulanmalıdır.

3. Malzeme Listesi ve Minimum Gereksinimler

3.1 Gate / Kapı Kontrol Ünitesi

Malzeme	Adet	Görev	Not
ESP8266 NodeMCU / Wemos D1	1	OpenPLC Ladder programını çalıştırır ve Modbus TCP server olur.	OpenPLC Editor'da ESP8266 NodeMCU target ile programlandı.
4 kanal 5V röle modülü	1	READY, REQUEST, PASS_ALLOWED, EMERGENCY durumlarını fiziksel olarak gösterir.	Active-low çalıştığı için Ladder çıkışları ters mantıkla bağlandı.
SG90 mikro servo	1	Bariyer/kapı mekanizmasını açıp kapatır.	D7 / GPIO13 üzerinden 50 Hz PWM ile sürüldü.
Jumper kablolar	Yeterli	ESP-röle-servo bağlantıları.	GND ortak olmalı.
USB kablo	1	Programlama ve besleme.	OpenPLC upload için kullanılır.
5V güç kaynağı	1	Röle ve servo beslemesi.	Servo akım çektiği için harici 5V önerilir.

Malzeme	Adet	Görev	Not
Mini bariyer mekanizması	1	Servo hareketini kapı hareketine çevirir.	Demo için hafif mekanizma kullanılmalıdır.

3.2 Jülide Robot

Malzeme	Adet	Görev	Not
ESP8266 NodeMCU / Wemos D1	1	Robot firmware'ini çalıştırır, web joystick sunar, gate ile Modbus TCP haberleşir.	Arduino IDE ile programlandı.
L298N motor sürücü	1	4 TT motorun yön ve hız kontrolünü sağlar.	Sol ve sağ motor grupları ayrı kontrol edildi.
TT DC motor	4	Robot hareketini sağlar.	Sol iki motor ve sağ iki motor grup olarak sürülür.
Tekerlek / mecamum tekerlek	4	Robot hareket mekanizması.	Prototip şaseye göre seçildi.
HC-SR04 mesafe sensörü	1	Kapı/engel mesafesini ölçer.	10 cm altında yavaşlama, 5 cm altında durma.
Batarya / pil grubu	1 set	Motor ve robot beslemesi.	Motorlar ESP'den doğrudan beslenmemelidir; GND ortak olmalıdır.
3D baskı / şase	1	Robot gövdesi.	Jülide prototipi için kullanıldı.

3.3 PC minimum gereksinimleri

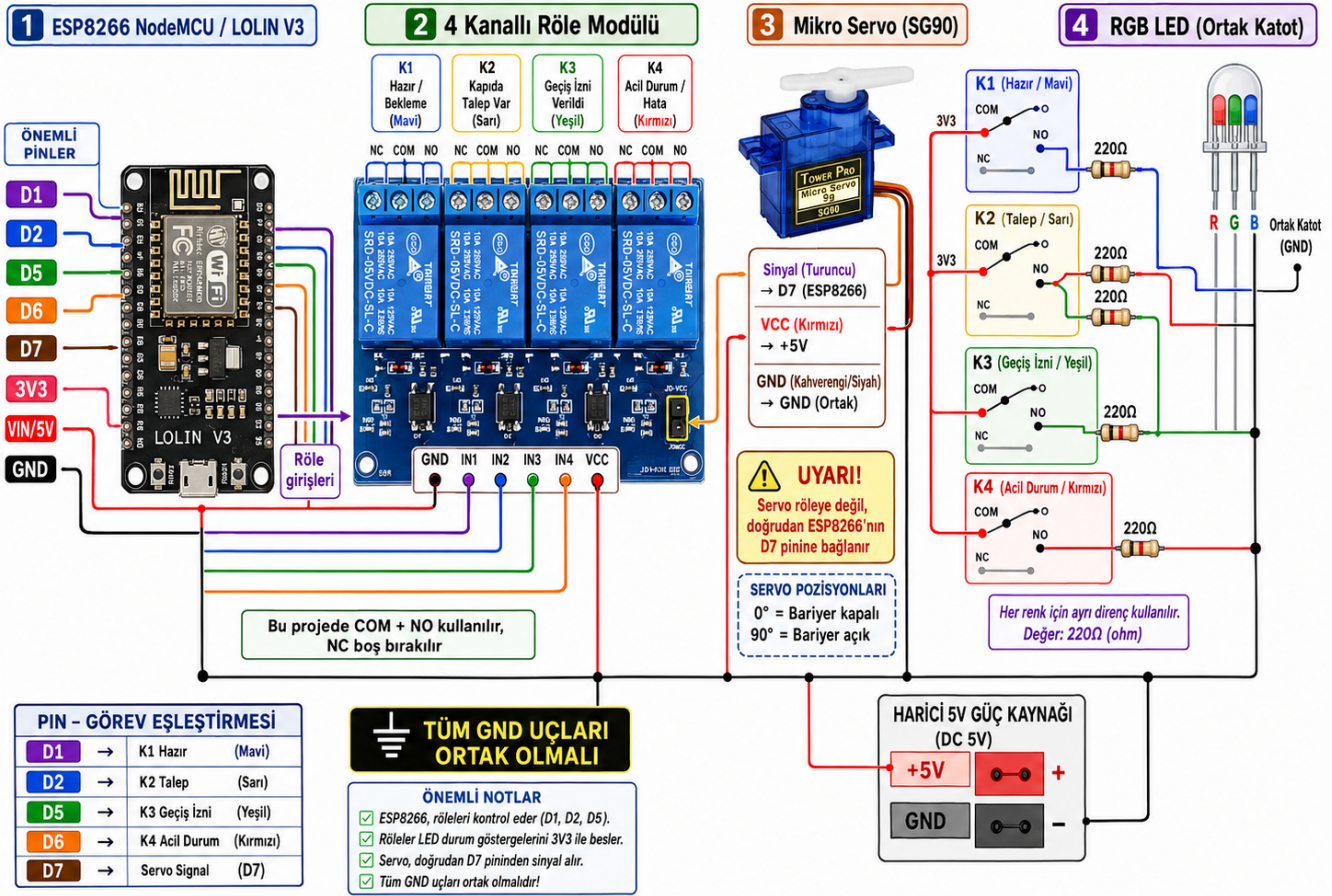
Gereksinim	Minimum	Önerilen / Not
İşletim sistemi	Windows 10/11	Windows Mobile Hotspot kullanılabiliyor olmalı.
RAM	4 GB	8 GB ve üzeri daha rahat.
Wi-Fi	Var	Hotspot açmak ve ESP'leri aynı ağa almak için gerekli.
USB port	1	ESP8266 programlamak için.
IDE/Yazılım	OpenPLC Editor, Arduino IDE, Python 3.10+	GUI için VS Code/Cursor, PySide6, pymodbus.
Python paketleri	PySide6, pymodbus, requests	pip install PySide6 pymodbus requests

4. Gate ESP8266 Donanım Bağlantıları

Gate tarafı OpenPLC Ladder programını çalıştırır. D1, D2, D5 ve D6 pinleri röle modülü girişlerine bağlıdır. D7 / GPIO13 servo sinyal pinidir. Röleler sistem durumunu fiziksel olarak gösterir; servo ise bariyer hareketini sağlar.

OpenPLC ESP8266 + 4'lü Röle + Servo + RGB LED Bağlantı Şeması

Servo doğrudan ESP8266'ya bağlanır, röleler durum göstergesi için kullanılır



Şekil 5 - OpenPLC ESP8266 + 4 kanal röle + servo bağlantı şeması. RGB LED kısmı opsiyoneldir; final projede röleler durum göstergesi olarak kullanılmıştır.

4.1 Final pin eşleşmesi

Bileşen	ESP8266 pini	GPIO	Açıklama
Röle K1 READY	D1	GPIO5	READY durum göstergesi
Röle K2 REQUEST	D2	GPIO4	Robot kapıda / izin bekliyor
Röle K3 PASS_ALLOWED	D5	GPIO14	Geçiş izni / kapı açık
Röle K4 EMERGENCY	D6	GPIO12	Acil durum / hata
Servo signal	D7	GPIO13	50 Hz PWM servo kontrol
Servo VCC	5V / VIN	-	Servo beslemesi
Servo GND	GND	-	Ortak toprak

4.2 Active-low röle mantığı

Kullanılan röle modülü active-low çalışır. Bu nedenle OpenPLC çıkışı FALSE/LOW olduğunda röle aktif olur. Fiziksel röle çıkışlarının beklenen state ile doğru yanması için Ladder'da Q çıkışları ters contact ile sürülmüştür.

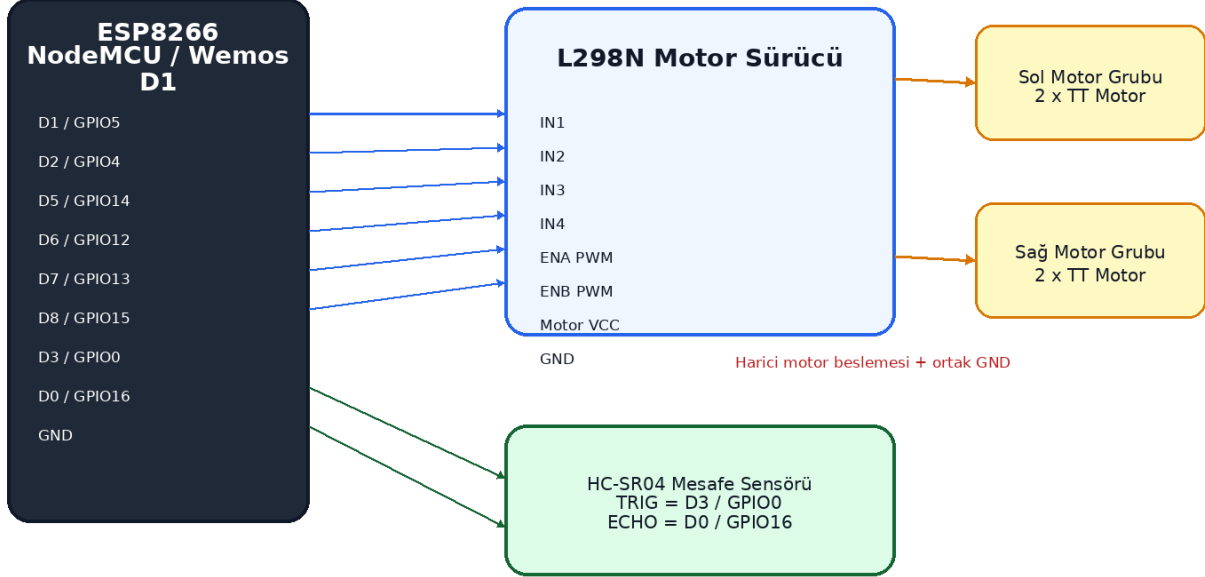
```

/ M_READY      -> Q_READY
/ M_REQUEST    -> Q_REQUEST
/ M_PASS_ALLOWED -> Q_PASS_ALLOWED
/ M_EMERGENCY  -> Q_EMERGENCY
    
```

5. Jülide Robot Donanım Bağlantıları

Jülide robot Arduino IDE ile programlanan ikinci ESP8266 kartı üzerinde çalışır. Bu kart L298N motor sürücüyü ve HC-SR04 mesafe sensörünü kontrol eder. Aynı zamanda HTTP web joystick arayüzü sunar ve Gate ESP'ye Modbus TCP client olarak bağlanır.

Jülide Robot Elektronik Bağlantı Özeti



Şekil 6 - Jülide robot elektronik bağlantı özeti

5.1 Jülide pin eşleşmesi

Bileşen	ESP8266 pini	GPIO	Açıklama
L298N IN1	D1	GPIO5	Sol motor yön 1
L298N IN2	D2	GPIO4	Sol motor yön 2
L298N IN3	D5	GPIO14	Sağ motor yön 1
L298N IN4	D6	GPIO12	Sağ motor yön 2
L298N ENA	D7	GPIO13	Sol motor PWM
L298N ENB	D8	GPIO15	Sağ motor PWM
HC-SR04 TRIG	D3	GPIO0	Mesafe sensörü trigger
HC-SR04 ECHO	D0	GPIO16	Mesafe sensörü echo

5.2 Jülide çalışma parametreleri

Parametre	Değer	Açıklama
motorSpeed	350 / 1023	Normal sürüş hızı
slowSpeed	220 / 1023	Engel yakinken yavaşlama hızı
startBoost	430 / 1023	Motor kalkış boost değeri

Parametre	Değer	Açıklama
boostTime	80 ms	Boost süresi
slowDistanceCm	10 cm	Yavaşlama mesafesi
stopDistanceCm	5 cm	Durma ve gate request mesafesi

6. Yazılım Bileşenleri ve IDE Ayrımı

Proje üç farklı geliştirme ortamı kullanır. Gate tarafı PLC mantığı için OpenPLC Editor'da hazırlanır; Jülide robot normal Arduino C++ firmware olarak Arduino IDE ile yüklenir; PC GUI ise Python/PySide6 ile VS Code veya Cursor üzerinde çalıştırılır.

Bileşen	IDE	Dil/Teknoloji	Görev
Gate ESP	OpenPLC Editor	Ladder Diagram	PLC state machine, Modbus TCP server, röle ve servo kontrolü
Jülide ESP	Arduino IDE	Arduino C++ / ESP8266	Motor kontrolü, HC-SR04 okuma, Modbus client, web joystick
PC GUI	VS Code / Cursor	Python + PySide6	HMI, operatör kontrolü, allow/reset/emergency, monitoring

6.1 Neden bu ayırım kullanıldı?

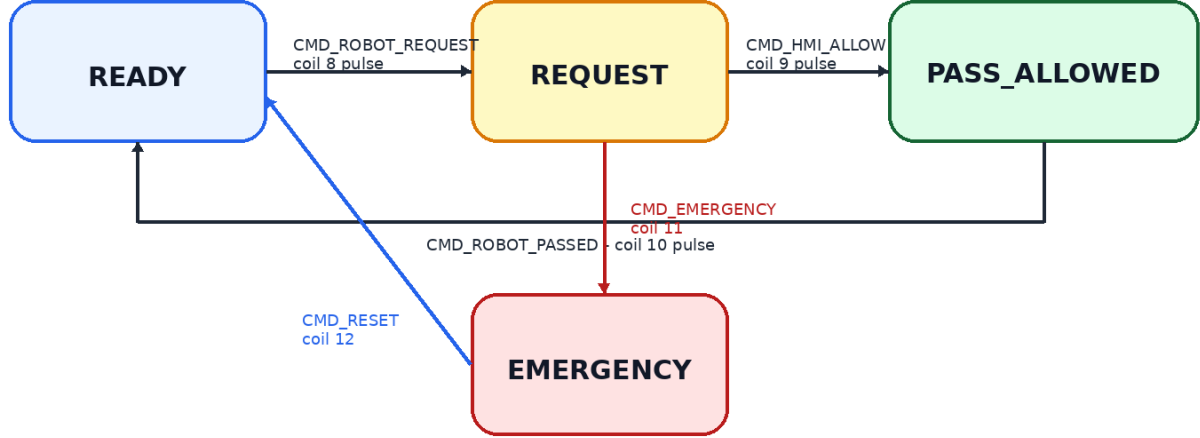
- Gate tarafında gerçek PLC mantığı göstermek için OpenPLC Ladder kullanıldı.
- Robot tarafında motor ve sensör kontrolü daha doğal olduğu için Arduino C++ kullanıldı.
- PC tarafında kullanıcı arayüzü ve test esnekliği için Python/PySide6 tercih edildi.
- Haberleşme protokolü olarak PLC dünyasına uygun Modbus TCP kullanıldı.

7. OpenPLC Ladder State Machine

Gate ESP üzerindeki Ladder programı, dört ana durumu tutar: READY, REQUEST, PASS_ALLOWED ve EMERGENCY. Durum geçişleri pulse komut coil'leri ile tetiklenir. Bu yapı, hem robotun hem de GUI'nin aynı PLC state machine üzerinde çalışmasını sağlar.

Gate OpenPLC State Machine

Durumlar rölelerle gösterilir: K1 READY, K2 REQUEST, K3 PASS_ALLOWED, K4 EMERGENCY.
Servo davranışı finalde state geçişiyle açıkça komutlandı: CLOSE -> OPEN -> CLOSE.



Şekil 7 - OpenPLC Ladder state machine

7.1 Durumlar

State	Fiziksel gösterge	Servo	Anlam
READY	K1	Kapalı	Sistem hazır; robot beklenir.
REQUEST	K2	Kapalı	Robot kapıda, operatör izni beklenir.
PASS_ALLOWED	K3	Açık	Geçiş izni verildi; robot geçebilir.
EMERGENCY	K4	Güvenli moda alınır	Acil durum; robot durmalıdır.

7.2 Ladder geçiş mantığı

```
CMD_RESET -> (S) M_READY
CMD_RESET -> (R) M_REQUEST
CMD_RESET -> (R) M_PASS_ALLOWED
CMD_RESET -> (R) M_EMERGENCY

CMD_EMERGENCY -> (S) M_EMERGENCY
CMD_EMERGENCY -> (R) M_READY
CMD_EMERGENCY -> (R) M_REQUEST
CMD_EMERGENCY -> (R) M_PASS_ALLOWED

M_READY + CMD_ROBOT_REQUEST -> (S) M_REQUEST
M_READY + CMD_ROBOT_REQUEST -> (R) M_READY

M_REQUEST + CMD_HMI_ALLOW -> (S) M_PASS_ALLOWED
M_REQUEST + CMD_HMI_ALLOW -> (R) M_REQUEST

M_PASS_ALLOWED + CMD_ROBOT_PASSED -> (S) M_READY
M_PASS_ALLOWED + CMD_ROBOT_PASSED -> (R) M_PASS_ALLOWED
```

8. Modbus TCP Coil Haritası

Final çalışan sistemde OpenPLC Modbus adresleri test edilerek doğrulandı. İlk denemelerde yüksek adresler veya farklı mapping beklentileri hata verdiği için finalde coil 0-12 aralığı kullanıldı.

8.1 Fiziksel çıkışlar

Değişken	OpenPLC location	Modbus coil	Fiziksel karşılık
Q_READY	%QX0.0	0	K1 READY
Q_REQUEST	%QX0.1	1	K2 REQUEST
Q_PASS_ALLOWED	%QX0.2	2	K3 PASS_ALLOWED
Q_EMERGENCY	%QX0.3	3	K4 EMERGENCY

8.2 Status bitleri

Değişken	OpenPLC location	Modbus coil	Anlam
STATUS_READY	%QX0.4	4	READY state bilgisi
STATUS_REQUEST	%QX0.5	5	REQUEST state bilgisi
STATUS_PASS_ALLOWED	%QX0.6	6	PASS_ALLOWED state bilgisi
STATUS_EMERGENCY	%QX0.7	7	EMERGENCY state bilgisi

8.3 Komut bitleri

Değişken	OpenPLC location	Modbus coil	Kim gönderir?	Görev
CMD_ROBOT_REQUEST	%QX1.0	8	Jülide / GUI debug	Robot kapıda, geçiş istiyor.
CMD_HMI_ALLOW	%QX1.1	9	GUI	Operatör geçiş izni verir.
CMD_ROBOT_PASSED	%QX1.2	10	Jülide / GUI debug	Robot geçti, kapı kapanabilir.
CMD_EMERGENCY	%QX1.3	11	GUI	Acil durum.
CMD_RESET	%QX1.4	12	GUI	Sistemi READY'ye döndürür.

8.4 Pulse mantığı

Komut bitleri kalıcı state tutmaz; pulse olarak kullanılır. Client coil'i TRUE yapar, kısa süre bekler ve tekrar FALSE yapar. Böylece Ladder tek bir kenar/olay gibi davranır.

```
write coil TRUE
wait 300 ms
write coil FALSE
```

9. Servo PWM Entegrasyonu

Servo normal dijital çıkış gibi sürülemez. SG90 tipi servo 50 Hz PWM sinyali ister. Bu nedenle OpenPLC Ladder içerisinde Arduino Function Blocks altındaki PWM_CONTROLLER bloğu kullanıldı. Ancak ESP8266 target tarafında bu blok için gerekli donanım PWM fonksiyonu eksikti.

9.1 Alınan compile hatası

```
undefined reference to `set_hardware_pwm`
```

Bu hata, PWM_CONTROLLER bloğunun OpenPLC ESP8266 derleme ortamında set_hardware_pwm fonksiyonunu bulamadığını gösterdi. Çözüm olarak build/src içine özel bir patch dosyası eklendi.

9.2 Eklenen patch

```
extern "C" uint8_t set_hardware_pwm(uint8_t ch, float freq, float duty)
{
```

```

uint8_t pin = ch;

if (ch == 7) {
    pin = 13;
}

if (pin != 13) {
    return 0;
}

if (freq < 1.0f) {
    freq = 50.0f;
}

if (duty < 0.0f) duty = 0.0f;
if (duty > 100.0f) duty = 100.0f;

pinMode(pin, OUTPUT);
analogWriteRange(1023);
analogWriteFreq((uint32_t)freq);

uint16_t pwmValue = (uint16_t)((1023.0f * duty / 100.0f) + 0.5f);
analogWrite(pin, pwmValue);

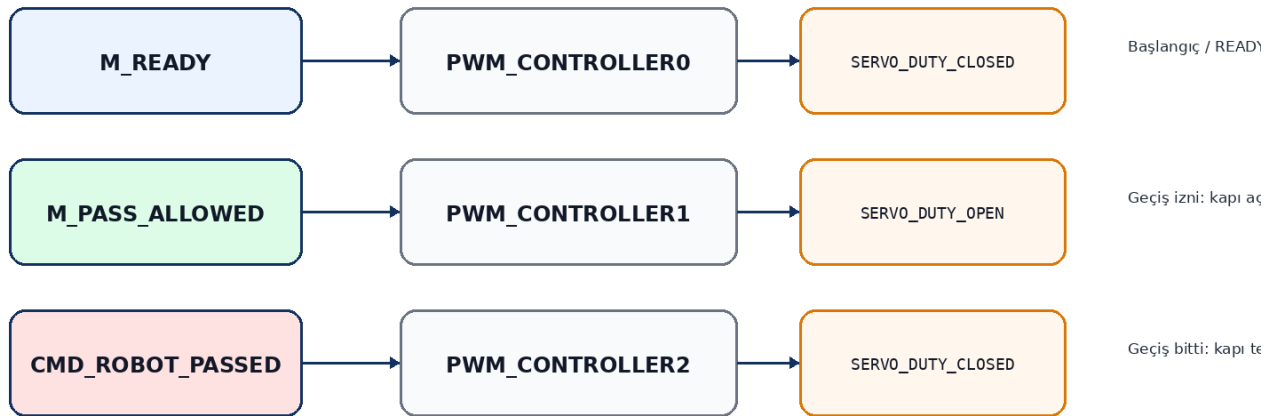
return 1;
}

```

Patch dosyası finalde build klasöründeki src içine yerleştirildi. Böylece OpenPLC derlemesi set_hardware_pwm fonksiyonunu linkleyebildi ve PWM_CONTROLLERER bloğu ESP8266 üzerinde çalıştı.

9.3 Final servo ladder mantığı

Servo Kontrolünün Final Mantığı



Not: Aynı servo kanalına final state olaylarında tekrar PWM yazmak, açıkta kalma sorununu çözdü.

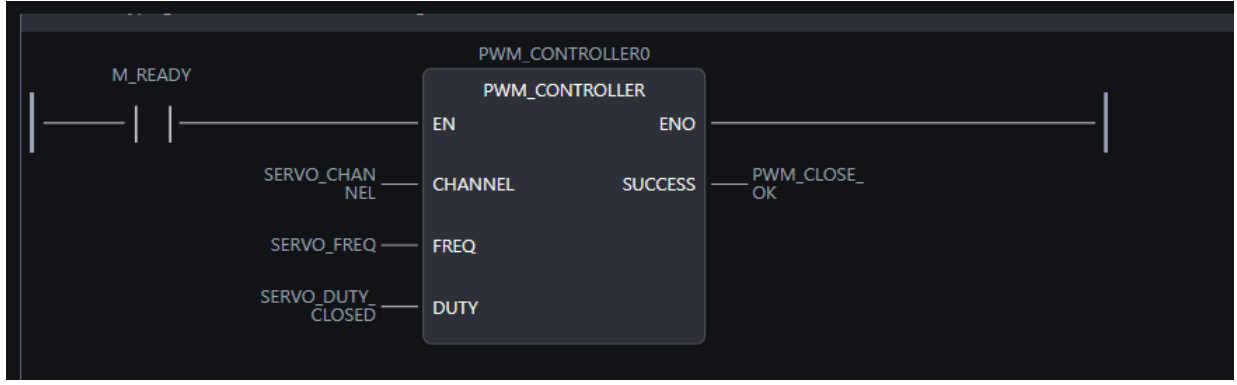
Şekil 8 - Final servo mantığı: SERVO CLOSE -> SERVO OPEN -> SERVO CLOSE

İlk denemede servo açılıyor fakat geçiş tamamlandıktan sonra kapanmıyordu. Sorunun nedeni, geçiş tamamlanma olayına açıkça tekrar servo close PWM komutunun bağlanmamış olmasıydı. Final çözümde üç PWM_CONTROLLERER komutu kullanıldı:

```

M_READY          -> PWM_CONTROLLER0 -> SERVO_DUTY_CLOSED
M_PASS_ALLOWED    -> PWM_CONTROLLER1 -> SERVO_DUTY_OPEN
CMD_ROBOT_PASSED  -> PWM_CONTROLLER2 -> SERVO_DUTY_CLOSED

```



Şekil 9 - OpenPLC içinde PWM_CONTROLLER blok kullanım örneği

9.4 Servo değişkenleri

Değişken	Tip	Başlangıç değeri	Açıklama
SERVO_CHANNEL	SINT	13	D7 / GPIO13 servo sinyal kanalı
SERVO_FREQ	REAL	50.0	SG90 için 50 Hz PWM
SERVO_DUTY_CLOSED	REAL	5.0	Kapalı pozisyon duty
SERVO_DUTY_OPEN	REAL	7.5	Açık pozisyon duty
PWM_CLOSE_INIT_OK	BOOL	False	İlk close komutu success
PWM_OPEN_OK	BOOL	False	Open komutu success
PWM_CLOSE_FINAL_OK	BOOL	False	Geçiş sonrası close komutu success

10. Jülide Robot Çalışma Mantığı

Jülide robotun görevi kapıya yaklaşmak, mesafe sensörüyle durmak, OpenPLC gate'e request göndermek, geçiş iznini beklemek ve geçiş tamamlandığında passed komutu göndermektir. Jülide ayrıca kullanıcıya web joystick arayüzü sunar.

10.1 Başlangıç akışı

1. ESP8266 açılır.
2. PC hotspot ağına bağlanır.
3. Serial Monitor'a IP adresi yazdırılır.
4. HTTP web joystick başlatılır.
5. Gate IP tanımlıdır; gerektiğinde Modbus TCP socket açılır.

10.2 Gate workflow

- Robot ileri sürülür
- > HC-SR04 mesafeyi okur
 - > 10 cm altında yavaşlar
 - > 5 cm altında durur
 - > Gate Mode açıksa CMD_ROBOT_REQUEST gönderir
 - > STATUS_PASS_ALLOWED beklenir
 - > izin gelince robot geçer
 - > CMD_ROBOT_PASSED gönderilir

10.3 Web joystick

Komut	Davranış
forward / f	Robot ileri gider.
backward / b	Robot geri gider.

Komut	Davranış
left / l	Robot sola döner.
right / r	Robot sağa döner.
stop / s	Motorlar durur.
Gate Mode	Otomatik request/pass akışını etkinleştirir.
Request	Manuel CMD_ROBOT_REQUEST pulse gönderir.
Passed	Manuel CMD_ROBOT_PASSED pulse gönderir.

10.4 Raw Modbus yaklaşımı

ModbusIP_ESP8266 kütüphanesi ile bağlantı kararsız kaldığında Jülide firmware'i raw Modbus TCP paketleri gönderecek şekilde sadeleştirildi. Bu sayede sadece gereken iki Modbus fonksiyonu kullanıldı: Function 01 Read Coils ve Function 05 Write Single Coil.

Function 01 - Read Coils

```
STATUS_READY      coil 4
STATUS_REQUEST    coil 5
STATUS_PASS_ALLOWED coil 6
STATUS_EMERGENCY  coil 7
```

Function 05 - Write Single Coil

```
CMD_ROBOT_REQUEST coil 8
CMD_ROBOT_PASSED  coil 10
```

11. GUI / HMI Çalışma Mantığı

PC üzerindeki PySide6 GUI, operatör arayüzüdür. Gate ESP'ye Modbus TCP client olarak bağlanır, status coil'lerini okur ve Allow/Emergency/Reset komutlarını pulse olarak gönderir.

11.1 GUI komutları

GUI işlemi	Coil	Açıklama
Allow	9	REQUEST durumundaki robota geçiş izni verir.
Emergency	11	Sistemi EMERGENCY durumuna alır.
Reset	12	Sistemi READY durumuna döndürür.
Simulate Robot Request	8	Test için robot request simülasyonu.
Simulate Robot Passed	10	Test için robot passed simülasyonu.

11.2 GUI status okuma

Status	Coil	GUI'deki anlam
STATUS_READY	4	Sistem hazır.
STATUS_REQUEST	5	Robot geçiş izni bekliyor.
STATUS_PASS_ALLOWED	6	Geçiş izni verildi.
STATUS_EMERGENCY	7	Acil durum.

11.3 Bağlantı yönetimi

ESP8266 OpenPLC tarafında aynı anda birden fazla Modbus client bağlantısı kararsızlık oluşturabili. GUI bağlantıyı sürekli açık tuttuğunda Jülide'nin socket açamadığı görüldü. Bu nedenle GUI tarafında Modbus işlemleri kısa bağlantı mantığına geçirildi.

```
connect
read/write
close
```

Bu değişiklikten sonra GUI ve Jülide aynı Gate ESP ile sırasıyla haberleşebildi.

12. Karşılaşılan Hatalar ve Çözümler

12.1 HTTP tabanlı ilk yaklaşım PLC gereksinimini karşılamıyordu

İlk yapı Arduino tarafında HTTP endpoint'leri ile çalışıyordu. Bu yapı robotik demo için yeterli olsa da PLC/Ladder gereksinimini doğrudan karşılamıyordu. Çözüm olarak gate tarafı OpenPLC Editor'a taşındı ve Ladder Diagram ile Modbus TCP coil tabanlı state machine kuruldu.

12.2 Rölelerin ters çalışması

Belirti: Beklenen state yerine farklı röleler yanıyor veya mantık ters görünüyordu. Sebep: Röle modülü active-low çalışıyordu. Çözüm: Fiziksel Q çıkışları Ladder'da ters contact ile sürüldü.

12.3 Yüksek Modbus adreslerinde Illegal Data Address

Başlangıçta %QX80.0 gibi yüksek adresler denenince Modbus exception_code=2 / Illegal Data Address görüldü. Çözüm: ESP8266 OpenPLC için pratik ve test edilmiş coil haritası 0-12 aralığında kuruldu.

12.4 Pymodbus Unit ID hatası

```
request ask for id=1 but got id=0
```

OpenPLC ESP8266 cevaplarında device id 0 kullanıldığı görüldü. Python/pymodbus tarafında device_id=0 ayarlanarak sorun çözüldü.

12.5 Jülide Modbus kütüphanesi bağlantı kararsızlığı

ModbusIP_ESP8266 ile bağlantı açma/okuma kısmı kararsız davrandı. Jülide firmware'i raw Modbus TCP paketleri gönderecek şekilde düzenlendi. Böylece bağımlılık azaldı ve sadece gereken Function 01/05 işlemleri kullanıldı.

12.6 Web joystick otomatik durma sorunu

Bazı ara sürümlerde güvenlik için otomatik stop eklendi, ancak manuel joystick hissini bozdu. Finalde joystick eski davranışına döndürüldü: yön komutu verildiğinde robot hareket eder, Dur komutuna kadar devam eder.

12.7 GUI'nin Modbus bağlantısını sürekli açık tutması

GUI bağlantıyı açık tuttuğunda Jülide'nin OpenPLC Modbus portuna erişimi zorlandı. Çözüm: GUI her işlemde connect-read/write-close yapacak şekilde düzenlendi; polling aralığı artırıldı.

12.8 Servo hiç çalışmıyordu

Sebep: Gate Ladder'da servo için PWM komutu yoktu. PWM_CONTROLLER bloğu eklendi; ancak derleme set_hardware_pwm fonksiyonu eksik olduğu için hata verdi.

12.9 PWM_CONTROLLER compile hatası

ESP8266 target tarafında set hardware_pwm fonksiyonu tanımlı olmadığı için undefined reference hatası alındı. openplc_esp8266_pwm_patch.cpp dosyası eklenerek analogWriteFreq ve analogWrite ile PWM üretimi sağlandı.

12.10 Servo açılıyor fakat kapanmıyordu

Final kritik hata buydu. K1 READY yanmasına rağmen servo açık kalıyordu. Bu, PLC state'inin doğru döndüğünü fakat geçiş tamamlandığında servo close komutunun açıkça tekrar yazılmadığını gösterdi. Çözüm: SERVO CLOSE / SERVO OPEN / SERVO CLOSE şeklinde üç PWM_CONTROLLER bloğu kullanıldı. Son close komutu CMD_ROBOT_PASSED sinyaline bağlandı.

13. Kurulum ve Çalıştırma

13.1 PC Hotspot

SSID : PLC_GATE_CTRL
Password : 12345678
Subnet : 192.168.137.x

13.2 Gate ESP / OpenPLC

1. OpenPLC Editor'da gate projesini aç.
2. Target: ESP8266 NodeMCU.
3. Wi-Fi SSID/password bilgilerini gir.
4. Pin mapping:
D1 -> %QX0.0 -> Q_READY
D2 -> %QX0.1 -> Q_REQUEST
D5 -> %QX0.2 -> Q_PASS_ALLOWED
D6 -> %QX0.3 -> Q_EMERGENCY
5. Servo patch dosyasının build/src içinde olduğundan emin ol.
6. Compile / Upload yap.
7. Başlangıçta K1 READY yanmalı ve servo kapalı olmalı.

13.3 Jülide

1. Arduino IDE'de Jülide final firmware dosyasını aç.
2. Board: NodeMCU / LOLIN Wemos D1 ESP8266.
3. Wi-Fi bilgilerini kontrol et.
4. Gate IP adresinin güncel olduğundan emin ol.
5. Upload et.
6. Serial Monitor'dan Jülide IP'sini al.
7. Tarayıcıdan http://JULIDE_IP/ adresine gir.

13.4 GUI / HMI

1. VS Code veya Cursor'da GUI klasörünü aç.
2. Python ortamını hazırla.
3. Gerekli paketleri kur:
pip install PySide6 pymodbus requests
4. Gate IP ve Jülide IP ayarlarını gir.
5. GUI'yi çalıştır.

14. Final Test Prosedürü

1. Gate ESP açılır.
Beklenen: K1 READY yanar, servo kapalıdır.
2. Jülide açılır.
Beklenen: Wi-Fi bağlantısı başarılıdır, web joystick erişilebilir.
3. Jülide ileri götürülür.

Beklenen: 10 cm altında yavaşlar, 5 cm altında durur.

4. Jülide request gönderir.
Beklenen: Gate K2 REQUEST yanar.
5. GUI üzerinden Allow verilir.
Beklenen: K3 PASS_ALLOWED yanar, servo açılır.
6. Jülide geçer.
Beklenen: Robot ileri hareket eder ve CMD_ROBOT_PASSED gönderir.
7. Geçiş tamamlanır.
Beklenen: K1 READY yanar, servo kapanır.
8. Emergency testi yapılır.
Beklenen: K4 EMERGENCY yanar, robot durur.
9. Reset gönderilir.
Beklenen: Sistem READY durumuna döner.

15. Troubleshooting

Belirti	Muhtemel sebep	Çözüm
Gate'e ping var ama Modbus bağlanmıyor	Yanlış port/IP veya OpenPLC çalışmıyor	IP doğrula, port 502 için Test-NetConnection çalıştır.
Jülide socket açamıyor	GUI Modbus bağlantısını açık tutuyor olabilir	GUI'yi kapat veya kısa bağlantı modunu kullan.
Röleler ters yanıyor	Active-low röle modülü	Q çıkışlarını ters contact ile sür.
Python Modbus id hatası	device_id uyumsuzluğu	pymodbus device_id=0 kullan.
Servo hiç hareket etmiyor	PWM patch yok veya sinyal pini yanlış	Patch dosyasını build/src içine koy; D7/GPIO13 bağlantısını kontrol et.
Servo açılıyor ama kapanmıyor	Geçiş sonrası close PWM komutu yok	CMD_ROBOT_PASSED -> PWM close bloğunu ekle.
Robot motorları zayıf dönüyor	Yetersiz motor beslemesi	L298N için uygun harici motor beslemesi kullan; GND ortakla.
HC-SR04 kararsız okuyor	Besleme veya pin sorunu	TRIG/ECHO pinlerini kontrol et; kabloyu kısalt; GND ortakla.

16. Güvenlik ve Güç Notları

- Motor, servo ve röle gibi yükler ESP8266 pinlerinden doğrudan beslenmemelidir; ESP yalnızca kontrol sinyali vermelidir.
- Servo ve motorlar akım çektiğinde ESP reset atabilir. Harici 5V besleme ve ortak GND önerilir.
- L298N motor beslemesi robot motorlarına uygun olmalıdır; USB üzerinden motor sürülmemelidir.
- Röle modülü active-low olduğu için PLC mantığı fiziksel çıkışlarla mutlaka test edilmelidir.
- Emergency komutu her zaman robotu durduracak şekilde ele alınmalıdır.
- Modbus TCP yerel ağ protokolüdür; üretim ortamında güvenlik katmanı düşünülmelidir.

17. Ders Çıkarımları

- PLC projesinde sadece çalışan Arduino kodu değil, state machine ve Ladder mantığı da açık olmalıdır.
- Modbus adres haritası varsayılmamalı; gerçek cihaz üzerinde test edilerek doğrulanmalıdır.

- ESP8266 gibi küçük kartlarda aynı anda çoklu client bağlantıları sorun çıkarabilir; bağlantı yönetimi önemlidir.
- Active-low röle modülleri Ladder çıktılarında ters mantık gerektirir.
- Servo kontrolü dijital çıkış değil PWM gerektirir; OpenPLC target tarafındaki donanım fonksiyonları eksikse patch gerekebilir.
- Bir state değişti diye fiziksel aktüatörün otomatik doğru konuma döneceği varsayılmamalıdır; gerekli komut açıkça yazılmalıdır.
- Final servo hatasında doğru çözüm, geçiş sonunda CMD_ROBOT_PASSED olayına tekrar close PWM komutu bağlamak oldu.

18. Teknik Özet

Final prototipte PC, Windows Mobile Hotspot ile ağı oluşturur. Gate ESP8266 OpenPLC Ladder programını çalıştırır ve Modbus TCP server olarak röleleri/servoyu kontrol eder. Jülide robot Arduino C++ firmware ile motorları ve HC-SR04 sensörünü yönetir, gate'e Modbus TCP client olarak request/passed komutları gönderir. PC GUI PySide6 ile operatörün Allow, Emergency ve Reset işlemlerini yapmasını sağlar.

Sistem finalde şu ana kural ile doğru çalışmıştır: READY durumunda kapı kapalı, REQUEST durumunda izin beklenir, PASS_ALLOWED durumunda servo açılır, CMD_ROBOT_PASSED geldiğinde servo tekrar kapanır ve sistem READY'ye döner.